CiT Backend Engineering Track

Week 7: Building APIs with Spring Boot

REST, Spring Boot, Controllers & Your First Web API

Detail	Information
Programme	CiT Backend Engineering Track
Phase	Phase 1 — Foundation (Weeks 1–8)
Week	Week 7
Instructor	Jim & Anderson
Duration	6 months | 24 weeks
Tech Stack	Java 21, Spring Boot 3, PostgreSQL, IntelliJ IDEA, Postman

Confidential — For CiT Trainees Only

Opening Prayer

We begin, as always, by acknowledging that every gift of understanding comes from God. Please bow your heads as we invite His presence into this session.

Prayer of Dedication

Heavenly Father,

Today we learn how the things we build speak to the wider world — sending requests, carrying messages, and delivering good news from one place to another. Teach us to carry information faithfully and truthfully, to connect people and systems with care, and to remember that behind every request is a real person we are called to serve.

As Proverbs 25:25 reminds us: "As cold water to a weary soul, so is good news from a far country." Lord, let the work of our hands bring something refreshing and good to those who receive it.

May we be diligent students who do not just copy but truly understand. May we encourage one another when we struggle. And may everything we build be offered as service to others.

In Jesus' name we pray,
Amen.

Scripture for the Week

"As cold water to a weary soul, so is good news from a far country."

— Proverbs 25:25 (NKJV)

Table of Contents

1. Welcome to Week 7

You have come a long way. You can write Java, design with objects, manage collections, handle errors, and store data permanently in a database. But everything you have built so far runs on your own machine, for your own eyes. This week, your work learns to talk to the outside world.

We will build a web API — a service that other programs can send requests to over the internet and get data back. This is the heart of modern backend engineering. Every mobile app, every website, every payment integration you have ever used talks to an API just like the one you will build today.

We use Spring Boot, the most popular framework for building Java backends in the world. And there is a special connection this week: the Frontend track is learning to consume APIs at the very same time. The service you build is exactly the kind of service they will fetch from. The two halves of an application are meeting.

What to Expect in Week 7

By the end of this session, you will be able to:

- Explain what an API is and what makes an API RESTful
- Describe what Spring Boot does and why frameworks save enormous time
- Create and run a Spring Boot application from scratch
- Build REST endpoints with @RestController and @GetMapping
- Accept input with @PostMapping, @PathVariable, and @RequestBody
- Connect your API to PostgreSQL using Spring Data JPA
- Test your API end to end with Postman

2. What Is an API? And What Is REST?

Before building one, understand exactly what an API is and why the world settled on a style called REST.

2.1 What Is an API?

API stands for Application Programming Interface. Stripped of jargon, an API is a way for one program to ask another program for something, using an agreed set of rules.

Think of a restaurant. You (one program) do not walk into the kitchen and cook. You read a menu, tell the waiter what you want, and the waiter brings it back. The waiter is the API: a clear, agreed interface between you and the kitchen. You do not need to know how the kitchen works — only what you can order and what you will get back.

A web API works the same way over the internet. A frontend (or a mobile app) sends a request — "give me all students" — and your backend sends back the data. The two never need to know each other's internal details.

2.2 HTTP — The Language of Web Requests

Web APIs communicate using HTTP, the same protocol your browser uses for every web page. Each request has a method (a verb describing the intent) and a URL (the address of the thing). The four you will use map neatly onto the database operations you learned last week.

HTTP Method	Means	Database Equivalent
GET	Fetch data — read something	SELECT
POST	Create something new	INSERT
PUT	Update an existing thing	UPDATE
DELETE	Remove something	DELETE

2.3 What Is REST?

REST is a popular set of conventions for designing web APIs in a clean, predictable way. An API that follows them is called RESTful. The core idea: organise your API around resources (things, like students), addressed by URLs, acted on with the HTTP methods above.

Request	What It Does
GET /api/students	Get the list of all students
GET /api/students/2024001	Get the one student with that registration number
POST /api/students	Create a new student (data sent in the request body)
DELETE /api/students/2024001	Delete that student

2.4 JSON — The Data Format

APIs send data as JSON (JavaScript Object Notation) — a simple, universal text format that both Java and JavaScript understand. It looks like this:

```
{
  "name": "Alice Nakato",
  "regNumber": "2024001",
  "gpa": 3.75
}
```

Spring Boot converts your Java objects into JSON automatically when sending, and JSON back into Java objects when receiving. You rarely write JSON by hand — but you must recognise it, because it is the language every API speaks.

3. Introducing Spring Boot

You could build a web API using only plain Java, but it would take hundreds of lines of tedious setup — handling network connections, parsing HTTP, converting JSON, and more. Spring Boot does all of that for you so you can focus on your actual logic.

3.1 What Is a Framework?

A framework is a large, ready-made foundation of code that handles the common, repetitive parts of a kind of application, leaving you to fill in the parts unique to your project.

Think of building a house. You could mill your own timber and forge your own nails — or you could start with a pre-built frame, plumbing, and wiring already in place, and simply design the rooms. A framework is that pre-built structure. Spring Boot is the most trusted frame for building Java backends.

3.2 What Spring Boot Gives You

Feature	What It Means for You
Embedded web server	A server (Tomcat) is built in. Run your app and it is instantly online — nothing to install separately.
Automatic JSON conversion	Java objects become JSON and back, with no manual work.
Annotations	Short labels like @RestController wire everything together — no lengthy configuration.
Database integration	Spring Data JPA connects to PostgreSQL with almost no boilerplate (Section 7).
Sensible defaults	"Convention over configuration" — it just works out of the box, and you adjust only what you must.

3.3 What Is an Annotation?

You saw @Override in Week 5. An annotation is a label beginning with @ that you attach to a class or method to give Spring instructions. For example, @RestController tells Spring "this class handles web requests". You will use a handful of these, and each one quietly replaces dozens of lines of setup.

4. Your First Spring Boot Application

Let us create and run a real Spring Boot application. In minutes you will have a server running on your own machine.

4.1 Generating the Project

Spring provides a website, start.spring.io, that generates a ready-to-run project. Follow these steps:

1. Open start.spring.io in your browser
2. Choose: Maven or Gradle, Java, Spring Boot 3, Java version 21
3. Add dependencies: **Spring Web** (for REST APIs) and **Spring Data JPA** + **PostgreSQL Driver** (for the database later)
4. Click Generate, then open the downloaded project in IntelliJ

4.2 The Main Class

Spring Boot generates one special class that starts everything. It looks like this:

```
@SpringBootApplication
public class CitApiApplication {

   public static void main(String[] args) {
      SpringApplication.run(CitApiApplication.class, args);
   }
}
```

Part	Plain English
@SpringBootApplication	One annotation that switches on all of Spring Boot's machinery — server, auto-configuration, and scanning for your classes.
SpringApplication.run(...)	Starts the whole application: boots the embedded web server and gets it listening for requests.
main(String[] args)	The same Java entry point you learned in Week 3 — Spring Boot is still just a Java program.

4.3 Running It

Press the green Run button. In the console you will see Spring Boot start up, ending with a line like "Tomcat started on port 8080". Your application is now live at http://localhost:8080. It does not do anything yet — that is the next section.

What is localhost and port 8080?

localhost means "this very computer" — your app is running on your own machine, not yet on the public internet. 8080 is the port: a numbered door on your computer where the server listens. The full address http://localhost:8080 means "knock on door 8080 of this computer".

5. Building a REST Controller — GET Endpoints

A controller is the class that handles incoming web requests. This is where your API actually comes to life. We will build endpoints that return student data.

5.1 Your First Endpoint

Create a class StudentController. The annotations do the heavy lifting.

```
@RestController
@RequestMapping("/api/students")
public class StudentController {

   @GetMapping
   public List<Student> getAllStudents() {
      return List.of(
         new Student("Alice Nakato", "2024001", 3.75),
         new Student("Bob Okello", "2024002", 3.10)
      );
   }
}
```

Run the app and open http://localhost:8080/api/students in your browser. You will see your two students returned as JSON. You just built a working web API.

5.2 Every Annotation Explained

Annotation	What It Does
@RestController	Marks this class as a handler of web requests whose return values are sent back as JSON.
@RequestMapping("/api/students")	Sets the base URL for every endpoint in this class. All of them start with /api/students.
@GetMapping	Maps HTTP GET requests for the base URL to this method. Visiting the URL runs getAllStudents().
return List.of(...)	Whatever you return is automatically converted to JSON by Spring Boot.

5.3 Path Variables — One Specific Item

To fetch one student by registration number, capture part of the URL with @PathVariable.

```
@GetMapping("/{regNumber}")
public Student getStudent(@PathVariable String regNumber) {
    // find and return the matching student
    return studentService.findByRegNumber(regNumber);
}
```

Now GET /api/students/2024001 captures 2024001 into the regNumber parameter. The {curly braces} in the path mark the changing part.

6. Handling Input — POST, Path Variables & Request Bodies

An API that only returns data is half a service. Real APIs also accept data — to create and change things. That is the job of POST requests and the request body.

6.1 Accepting New Data with @PostMapping

A POST request carries data in its body, as JSON. The @RequestBody annotation tells Spring to convert that incoming JSON into a Java object for you.

```
@PostMapping
public Student addStudent(@RequestBody Student student) {
   studentService.save(student);
   return student;   // send the saved student back as confirmation
}
```

When a client sends a POST to /api/students with a JSON body, Spring builds a Student object from it, hands it to your method, you save it, and you return it. The frontend will do exactly this when a user submits a form.

6.2 The Annotations for Input

Annotation	Where the Data Comes From
@PathVariable	A value taken from the URL path, e.g. the 2024001 in /api/students/2024001.
@RequestBody	The JSON body sent with a POST or PUT, converted into a Java object.
@RequestParam	A query parameter after a ? in the URL, e.g. /api/students?active=true.

6.3 The Full Set of Endpoints

Putting the HTTP methods together gives a complete, RESTful resource:

```
@GetMapping           // GET    /api/students       → list all
@GetMapping("/{reg}")   // GET    /api/students/{reg}  → one student
@PostMapping          // POST   /api/students       → create
```

```
@PutMapping("/{reg}")    // PUT    /api/students/{reg}  → update
@DeleteMapping("/{reg}") // DELETE /api/students/{reg}  → remove
```

Exercise 1: Design a courses Endpoint

On paper, design the five RESTful endpoints for a courses resource. For each, write the HTTP method, the URL, and one sentence describing what it does. Then write the Java method signature (with its annotation) for the GET-all and the POST endpoints.

7. Connecting the API to the Database with Spring Data JPA

Last week you stored students in PostgreSQL with raw SQL and JDBC. Spring Data JPA lets your API talk to that same database with almost no code at all — it writes the SQL for you.

7.1 The Entity — Mapping a Class to a Table

An entity is a Java class that maps directly onto a database table. Annotations connect the two: each object becomes a row, each field a column.

```
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String regNumber;
    private double gpa;

    // constructors, getters, and setters
}
```

Annotation	Meaning
@Entity	This class maps to a database table (named student by default).
@Id	This field is the primary key.
@GeneratedValue	The database generates the id automatically, like SERIAL last week.

7.2 The Repository — Database Access for Free

A repository is an interface that gives you ready-made database operations. You write an empty interface; Spring provides the implementation at runtime.

```
public interface StudentRepository extends JpaRepository<Student, Long> {
    // that is it — no code needed
}
```

By extending JpaRepository, you instantly get methods like findAll(), save(), findById(), and deleteById() — all writing correct SQL for you. No JDBC, no PreparedStatement, no ResultSet.

7.3 Using the Repository in the Controller

```
@RestController
@RequestMapping("/api/students")
public class StudentController {

    private final StudentRepository repository;

    public StudentController(StudentRepository repository) {
        this.repository = repository;   // Spring provides it for you
    }

    @GetMapping
    public List<Student> getAll() {
        return repository.findAll();    // reads from PostgreSQL
    }

    @PostMapping
    public Student create(@RequestBody Student student) {
        return repository.save(student);  // writes to PostgreSQL
    }
}
```

What is Dependency Injection?

Notice you never wrote new StudentRepository(). Spring creates it and hands it to your controller through the constructor. This is called dependency injection — the framework provides the objects your class needs. It keeps your classes focused and easy to test. You will study it more deeply in Phase 2.

8. Testing Your API with Postman

Your browser can only easily make GET requests. To test POST, PUT, and DELETE — sending data and choosing the method — you need a proper API testing tool. Postman is the industry standard, and it is free.

8.1 What Postman Does

Postman lets you send any HTTP request to your API and inspect the response. You choose the method, type the URL, attach a JSON body when needed, and see exactly what comes back — the data, the status code, and how long it took. It is how every professional backend engineer checks their work.

8.2 Testing a GET

5. Open Postman and create a new request
6. Set the method to GET and the URL to http://localhost:8080/api/students
7. Click Send. The JSON list of students appears in the response panel

8.3 Testing a POST — Sending Data

8. Set the method to POST and the same URL
9. Go to the Body tab, choose "raw", and select JSON
10. Type a new student as JSON:

```
{
  "name": "Carol Atim",
  "regNumber": "2024003",
  "gpa": 3.90
}
```

11. Click Send. Your API creates the student and returns it. Then GET the list again to confirm it was saved

8.4 HTTP Status Codes

Every response carries a status code — a number telling the client what happened. Recognise the common ones:

Code	Meaning
200 OK	Success — here is your data
201 Created	Success — a new resource was created (after a POST)
400 Bad Request	The client sent something invalid
404 Not Found	No resource exists at that URL
500 Server Error	Something broke inside your code

Project Milestone — Expose Your Students as an API

Bring the Student Grade Management System fully online:

12. Turn `Student` into a JPA `@Entity` and create a `StudentRepository`
13. Build a `StudentController` with GET-all, GET-one, and POST endpoints
14. Use Postman to add three students and fetch them back
15. Confirm the data persists in PostgreSQL after restarting the app

9. Key Takeaways, Homework & Exercises

9.1 What You Learned This Week

- An API is an agreed interface for one program to request something from another
- REST organises APIs around resources, addressed by URLs and acted on with HTTP methods (GET, POST, PUT, DELETE)
- JSON is the universal data format APIs use; Spring Boot converts to and from it automatically
- A framework like Spring Boot provides the ready-made foundation so you write only your own logic
- Building endpoints with @RestController, @GetMapping, @PostMapping, @PathVariable, and @RequestBody
- Spring Data JPA — entities and repositories give database access with almost no code
- Testing an API end to end with Postman, and reading HTTP status codes

9.2 Homework

16. Complete the in-class exercise and push your Spring Boot project to GitHub
17. Complete the project milestone: expose your Student Grade Management System as a REST API backed by PostgreSQL, with GET, POST, and DELETE endpoints
18. Add a courses resource to your API — an entity, a repository, and a controller with full CRUD endpoints — and test every endpoint in Postman
19. Preview Week 8: Think about how a frontend and a backend connect into one product. Come ready to combine the two halves of what the class has built.

Coming Next — Week 8: Phase 1 Capstone — Full-Stack Integration

Jim & Anderson will help you close Phase 1 by connecting the two tracks into one working product: a frontend that fetches from the backend API you built this week. You will see your eight weeks of foundation become a complete, end-to-end application — and prepare for Phase 2.

Closing Reflection

"As each one has received a gift, minister it to one another, as good stewards of the manifold grace of God."

— 1 Peter 4:10 (NKJV)

An API is, at its heart, an act of service — your code making itself useful to others. The skill you gained today lets your work serve people you will never meet. Build your services to be reliable, honest, and helpful, as good stewards of the abilities God has given you.

Glossary of Terms

Term	Definition
Annotation	A label beginning with @ that gives Spring instructions, e.g. @RestController
API	Application Programming Interface — an agreed way for programs to request things from each other
Controller	A class that handles incoming web requests and returns responses
CRUD	Create, Read, Update, Delete — the four basic operations on data
Dependency Injection	The framework creating and supplying the objects a class needs, rather than the class building them
Endpoint	A specific URL on an API that performs one operation
Entity	A Java class mapped to a database table with JPA annotations
Framework	A ready-made code foundation handling common tasks so you write only your own logic
HTTP	The protocol web requests use; each has a method (GET, POST, ...) and a URL
JSON	JavaScript Object Notation — the universal text format APIs use for data
JPA	Java Persistence API — maps Java objects to database tables; Spring Data JPA implements it
Postman	A tool for sending HTTP requests to an API and inspecting the responses
Repository	An interface giving ready-made database operations for an entity
REST	A set of conventions for designing APIs around resources and HTTP methods
Spring Boot	The most popular Java framework for building web applications and APIs
Status Code	A number in an HTTP response indicating the outcome, e.g. 200, 404, 500